

Planen, erkunden, lernen

Wie der Copilot Workflows bauen und optimieren sollte — Konzept v2

Erweiterte Fassung · 30. Juli 2026 · Diskussionsgrundlage, keine Implementierung

Version 1 schlug vor, Selektoren vorab zu prüfen und Bewährtes zu merken. Diese Fassung geht weiter und beantwortet drei Fragen: Wie kann der Copilot von erfolgreichen Workflows lernen und sie auf neue Portale übertragen? Wie kann er eine Seite selbst im Browser erkunden, statt nur zuzusehen? Und warum hat die vorhandene Bild- und DOM-Analyse das bisher nicht geleistet?

Die zentrale Einsicht dieser Fassung

Der Copilot hat heute fast alles, was er zum Handeln braucht: Er sieht die Seite, erkennt Elemente, kann Aktionen mit Element-Bezug vorschlagen. Was fehlt, ist der Regelkreis — er darf seine Vermutung nie sofort ausprobieren und die Wirkung messen. Stattdessen wandert sie ungeprüft in den Workflow und zeigt sich erst Minuten später im nächsten Lauf, wo Ursache und Wirkung nicht mehr zuzuordnen sind. Das ist der Kern des Problems.

Warum Bild- und DOM-Analyse bisher nichts getragen haben

Die Technik ist vorhanden und liefert brauchbare Ergebnisse. Sie scheitert an vier Konstruktionen:

- Das Modell darf urteilen, nicht handeln. Es liefert Aktionsvorschläge mit Element-Bezug (`suggested_task_actions`), aber ausgeführt wird daraus nur eine Workflow-Änderung — nie ein sofortiger Versuch mit Rückmeldung.
- Vermutung und Beweis werden nicht getrennt. „Das ist vermutlich der Ablehnen-Knopf“ und „dieser Knopf hat den Banner nachweislich geschlossen“ landen beide gleichwertig im Workflow.
- Sicherheitslogik aus dem Produktivlauf gilt auch beim Erkunden. Unsicherheit führt zu „pausieren“ — richtig, wenn echte Daten im Spiel sind; falsch, wenn der Copilot gerade herausfinden soll, wie die Seite funktioniert.
- Nichts wird behalten. Jede Erkenntnis stirbt mit der Sitzung, jeder neue Workflow beginnt bei null.

1. Drei Ebenen von Wissen

Damit der Copilot uebertragen kann, was anderswo funktioniert hat, muss Wissen getrennt gespeichert werden nach dem, was portalunabhaengig ist, und dem, was nur fuer eine Seite gilt.

R Rezept — was portalunabhaengig gilt

Die Abfolge der Schritte mit Rollen statt Selektoren: Consent behandeln → Suchfeld fuellen → absenden → auf Treffer warten → Treffer durchlaufen → Werte sammeln → Ergebnis zurueckgeben. Ein Rezept enthaelt keine einzige CSS-Zeile. Es entsteht automatisch aus jedem erfolgreich gelaufenen Workflow, indem dessen Selektoren durch die Rollen ersetzt werden, die das Vision-Modell den Elementen ohnehin schon gibt.

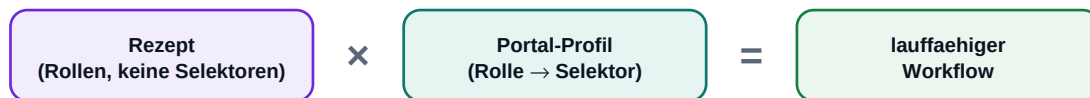
P Portal-Profil — was nur fuer diese Seite gilt

Pro Domain die Zuordnung von Rolle zu Selektor: google.com · Suchfeld → textarea[name="q"] · zuletzt bestaetigt am 30.07. · 12 von 12 Treffern. Dazu Besonderheiten der Seite: Erscheint ein Consent-Dialog? Braucht es eine Anmeldung? Das Profil entsteht beim Erkunden und wird durch jeden Lauf bestaetigt oder entwertet.

L Lauf-Historie — was tatsaechlich passiert ist

Die vorhandenen Ereignisprotokolle, aber ausgewertet statt nur archiviert: Welche Rezepte fuehren wie oft zum Ziel? Welche Selektoren halten laenger als andere? Welche Portale sind grundsaeztlich schwierig? Daraus entsteht die Rangfolge, in der der Copilot Vorschlaege macht.

Die Formel dahinter



Der praktische Nutzen: Ein Workflow, der auf Google funktioniert, wird auf Bing uebertragen, indem nur das Portal-Profil neu erkundet wird — das Rezept bleibt. Und ein neues Portal ohne Profil startet nicht bei null, sondern mit den Rollen, die das Rezept verlangt: Der Copilot weiss genau, wonach er suchen muss.

Warum Rollen und nicht der ganze Workflow kopiert wird

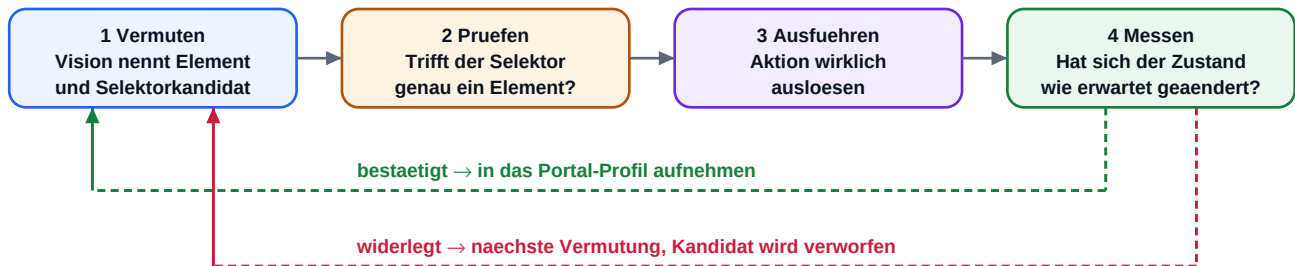
Ein fertiger Workflow laesst sich nicht uebertragen — seine Selektoren gelten nur fuer eine Seite. Die Schrittfolge dagegen ist erstaunlich stabil: „Consent, Suchfeld, absenden, sammeln, zurueckgeben“ gilt fuer jede Suchmaschine. Genau diese Trennung macht Uebertragbarkeit erst moeglich.

2. Der Erkundungsmodus — der Copilot fasst selbst an

Der zweite grosse Baustein: Der Copilot bekommt einen Modus, in dem er die Seite selbst bedienen darf, ohne dass ein Workflow laeuft. Nicht um Arbeit zu erledigen, sondern um herauszufinden, wie die Seite funktioniert.

Der Regelkreis auf Elementebene

Statt einer Vermutung, die ungeprueft in den Workflow wandert, laeuft jede Annahme durch vier Schritte — im Sekundenbereich, nicht ueber einen ganzen Lauf hinweg:



Der Unterschied zu heute ist gross: Eine widerlegte Vermutung kostet Sekunden statt einen ganzen Lauf, und eine bestaetigte ist ein Beweis — kein Verdacht. Nur bestaetigte Selektoren landen im Portal-Profil.

Was der Copilot beim Erkunden darf — und was nicht

Erkunden heisst handeln, und Handeln kann Schaden anrichten. Deshalb braucht jede Aktionsart eine klare Einstufung. Die Grenze verlaeuft entlang der Frage: Ist der Schritt umkehrbar?

- **Immer erlaubt**

Seite oeffnen, lesen, scrollen, Element anvisieren, DOM und Bild auswerten

- **Erlaubt mit Ruecksetzpunkt**

Klicks ohne Datenwirkung: Consent, Aufklappen, Tabs, Filter. Vorher Zustand sichern, danach zuruecksetzen

- **Nur mit Testdaten**

Formularfelder fuellen — ausschliesslich mit hinterlegten Testwerten, nie mit echten Zugangsdaten

- **Nur nach Freigabe**

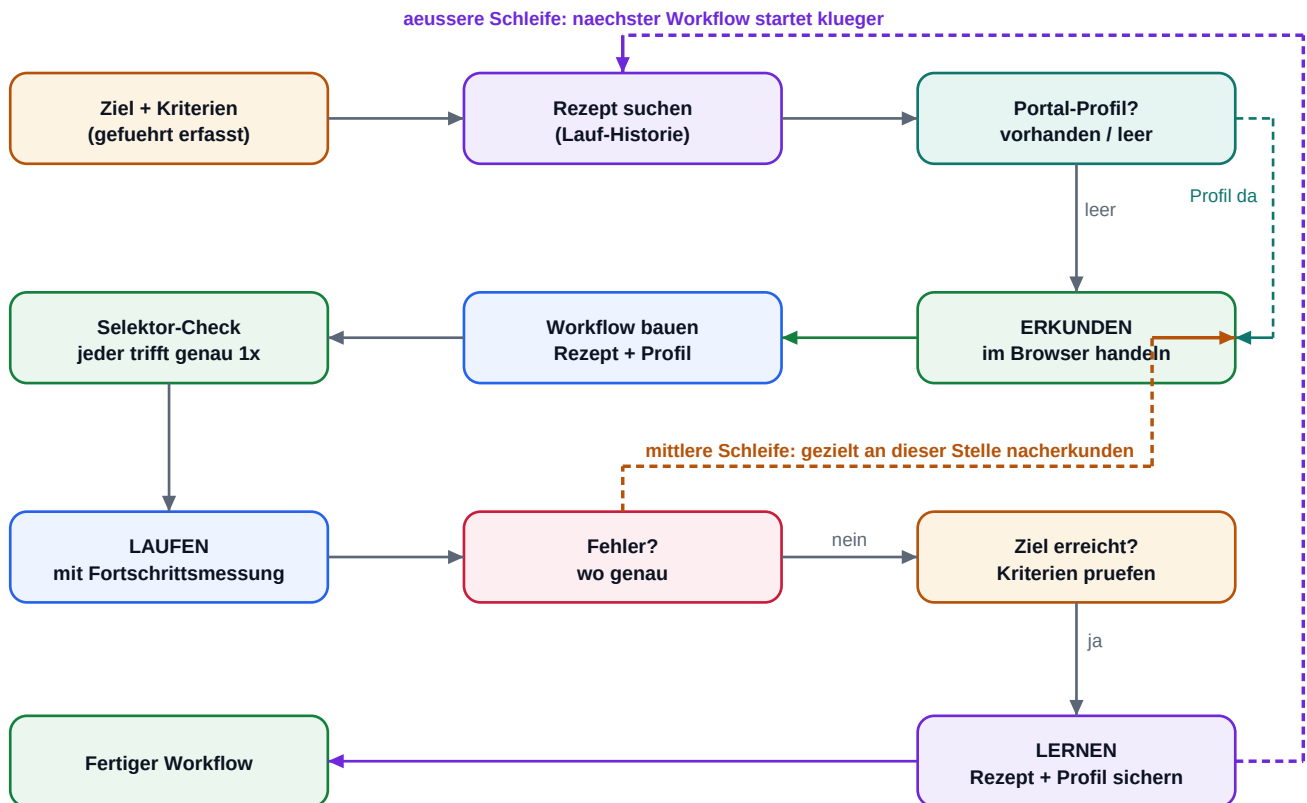
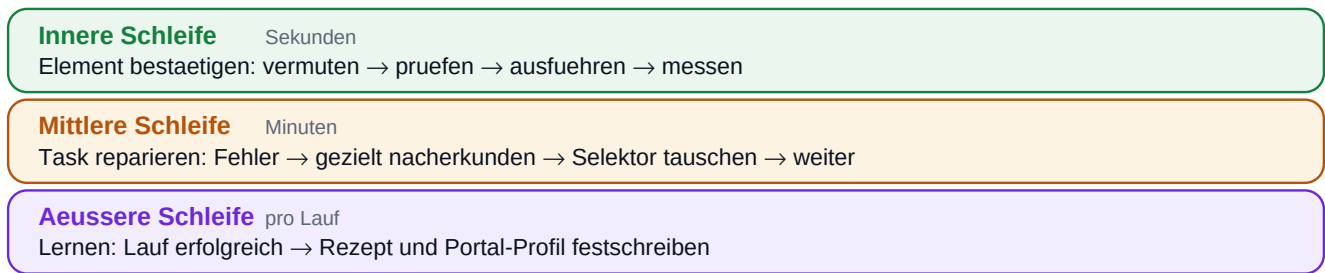
Absenden, anmelden, kaufen, loeschen, alles Bezahlte oder nach aussen Wirkende

Der wichtigste Nebeneffekt

Eine Anmeldung, die der Copilot nicht selbst durchfuehren darf, ist kein Hindernis — er erkundet bis zur Anmeldemaske, meldet „ab hier brauche ich dich“, und der Mensch macht einen Klick. Danach laeuft die Erkundung weiter. Das ist deutlich billiger als vierzig Minuten Reparaturschleife vor einem Login-Formular, das nie ausgefuellt wurde.

3. Der Ablaufplan mit seinen Schleifen

Alles zusammen ergibt drei ineinanderliegende Schleifen mit sehr unterschiedlicher Geschwindigkeit. Das ist der Kern des Konzepts.



Die innere Schleife steckt im Kasten ERKUNDEN und ist auf der vorigen Seite dargestellt. Wichtig ist der Unterschied zu heute: Bei einem Fehler wird nicht die Struktur umgebaut, sondern gezielt an genau der Stelle nacherkundet, an der es klemmt.

4. Umsetzung, Datenhaltung, Grenzen

Was technisch neu gebaut werden muss

- Zwei Tabellen. Portal-Profile (Domain, Rolle, Selektor, Trefferzahl, Fehlzahl, zuletzt bestaetigt) und Rezepte (Name, Schrittfolge mit Rollen, Erfolgsquote, Herkunftsworkflow). Beides klein und unabhaengig vom bestehenden Workflow-Modell.
- Ein Erkundungsmodus mit eigener Sicherheitsstufe — technisch eine Sitzungsart, die dieselben Browser-Tasks nutzt, aber nach der Umkehrbarkeits-Einstufung filtert statt nach dem Workflow-Korsett.
- Eine Rezept-Ableitung: Aus einem erfolgreichen Lauf die Schrittfolge lesen und Selektoren durch Rollen ersetzen. Die Rollenbenennung liefert das Vision-Modell bereits mit.
- Der Selektor-Check aus Version 1 als eigener Schritt vor dem Lauf.

Was bereits vorhanden ist und nur anders verwendet wird

- Die Interaktionskarte mit Selektorkandidaten pro Element — sie entsteht in jeder Beobachtung und wird nach dem Lauf verworfen. Sie ist die Rohdatenquelle fuer Portal-Profile.
- Aktionsvorschlaege mit Element-Bezug — der Copilot formuliert sie heute schon, darf sie aber nur als Workflow-Aenderung einreichen statt sie auszuprobieren.
- Der Zielfortschritt von 0 bis 1 — wird erhoben, protokolliert und dann ignoriert.
- Die Probe-Mechanik fuer einzelne Tasks — die halbe Miete fuer den Regelkreis, heute nur reaktiv im Reparaturfall.

Reihenfolge der Einfuehrung

Erst der Selektor-Check und die gefuehrte Zieleingabe — beide klein, sofort wirksam, ohne Eingriff in die Laufsteuerung. Dann die Portal-Profile, gefuellt aus der bereits vorhandenen Interaktionskarte. Dann der Erkundungsmodus, der die Profile aktiv fuellt statt nebenbei. Zuletzt Rezepte und Uebertragbarkeit — sie brauchen eine gewisse Menge erfolgreicher Laeufe, um ueberhaupt Material zu haben.

Abwaegungen, die vor einer Entscheidung auf den Tisch gehoeren

- Ein Erkundungsmodus, der klickt, kann etwas ausloesen, das nicht gewollt war. Die Umkehrbarkeits-Einstufung ist deshalb keine Formalie, sondern die tragende Sicherheitsentscheidung — sie sollte pro Task-Typ im Katalog hinterlegt und nicht vom Modell entschieden werden.
- Gespeicherte Selektoren veralten. Ohne Verfallsdatum und Fehlerzaehler wird das Gedaechnis irgendwann zur Fehlerquelle. Ein Eintrag, der zweimal hintereinander nicht trifft, sollte automatisch verfallen.
- Rezepte verleiten dazu, ein Muster auf eine Seite zu pressen, die anders funktioniert. Ein Rezept sollte immer nur Vorschlag sein und beim Selektor-Check widerlegbar bleiben.
- Mehr Automatik heisst mehr Erklaerbedarf. Wenn der Copilot kuenftig selbst klickt, muss im Protokoll unterscheidbar bleiben, was er vermutet, was er getestet und was ein Mensch bestaetigt hat.

Was auch dieses Konzept nicht loest

Seiten mit aktivem Bot-Schutz, Zwei-Faktor-Anmeldung oder staendig wechselndem Aufbau bleiben schwierig. Fuer sie ist die richtige Antwort nicht mehr Automatik, sondern ein frueher, klar begruendeter Abbruch — und die Moeglichkeit, den entscheidenden Schritt von Hand zu erledigen und den Rest wieder dem Copilot zu ueberlassen.

Der naechstliegende Praxistest

Den Google-Suche-Workflow nehmen, dessen problematische Selektoren aus den Protokollen bekannt sind. Erst den Selektor-Check darauf laufen lassen — findet er die bekannten Fehltreffer, ist Baustein A belegt. Danach dieselbe Aufgabe auf einer zweiten Suchmaschine: Traegt das Rezept, muss nur das Portal-Profil neu erkundet werden. Das ist der Beweis fuer Uebertragbarkeit, bevor groesser gebaut wird.